2024 International Conference on Intelligent Robotics and Automatic Control (IRAC)

Improved A* Algorithm and Dynamic Obstacle Avoidance Method for Path Planning and Self-Positioning of Autonomous Guided Vehicles

Yingrui Wang
Shien-Ming Wu School of Intelligent Engineering
South China University of Technology
Guangzhou, China

Junting Meng
School of Mechanical & Automotive Engineering
South China University of Technology
Guangzhou, China

Dong Zhang
School of Future Technology
South China University of Technology
Guangzhou, China

Xiaoqi Chen*
Shien-Ming Wu School of Intelligent Engineering
South China University of Technology
Guangzhou, China
* Corresponding author: xqc@scut.edu.cn

2024 International Conference on Intelligent Robotics and Automatic Control (IRAC) | 979-8-3503-8980-7/24/$31.00 ©2024 IEEE | DOI: 10.1109/IRAC63143.2024.10871658

***Abstract*—As Autonomous Guided Vehicles (AGV) are becoming more and more widely used, they demand safe and reliable autonomous navigation solutions to problems including how to plan a safe and efficient path based on known environmental information, how to accurately self-locate, and how to precisely avoid obstacles during the navigation process. These problems have given rise to path planning algorithms that have been the focus of research in the field of intelligent robot control. This paper develops new methods for path planning and self-positioning. Firstly, it develops the improved A* algorithm by constructing a heuristic function and increasing the weight of the heuristic cost. The improved algorithm significantly reduces the computation of the traversal of irrelevant node points and hence the computation time. Further it adds a smoothing process for unsmooth paths using Bessel curves, resulting in sufficiently smooth curve. Simulations of the algorithm show that the improved A* algorithm can significantly reduce the number of nodes traversed while ensuring optimal paths. Secondly, in terms of self-positioning, the distance between the visual tag and the robot is analyzed using the visual tag information acquired by the robot vision system to determine the position and orientation of the robot. To achieve collision avoidance during navigation in dynamic environments, a dynamic obstacle avoidance method is proposed, which combines TOF sensors to acquire local environment information; dynamic obstacle avoidance is achieved by inserting and switching tracking sub-targets. Finally, path planning tests were carried out using the AGV. Results show that the method can meet the needs of AGV applications in complex and dynamic environments.**

Keywords-Path planning; Self-positioning; A* algorithm; Dynamic obstacle avoidance; Autonomous Guided Vehicle (AGV)

I. Introduction

With the continuous improvement of science and technology, social labor productivity has also increased, to free human beings from simple and repetitive labor, people have designed robots that can help or even replace human beings to complete simple and repetitive or dangerous tasks [1]. With the further development of science and technology, robots have entered into various fields of human society. Robots are not only widely used in the service industry, such as service and shopping guide, but also play an unprecedented role in military, marine, space exploration and other fields [2,3]. With the rise of artificial intelligence technology, a large number of scholars and researchers have invested in research to combine artificial intelligence technology and robotics technology, which has greatly contributed to the development of a new generation of intelligent robots.

Mobile robots are one of the most commonly used robots and one of the most researched robots. The mobile robot path planning problem is also a really important research problem for mobile robots at present, which has important academic significance and practical value [4]. Mobile robot path planning more refers to the mobile robot using its own sensors to understand the surrounding environment, and according to a certain optimization goal, to find a safe, collision-free, smooth and most efficient optimal route from the starting point to the end point in the map. How the robot can accurately plan a safe and optimal path, how the robot can accurately determine its own position during autonomous navigation, and how to achieve real-time obstacle avoidance are the three major problems of path planning [5]. The result of path planning will have a great impact on the effectiveness of the mobile robot to accomplish the task.

Mobile robot path planning research has made some progress so far, but relatively speaking it is still far from enough. From the environmental situation in which the robot is located, path planning can be divided into two main aspects which are global planning and local planning. Global planning is to carry out path planning in the environment where the robot is located. All information about the environment, which is known before planning, is taken into consideration in the planning process, leading to a shortest or fastest path. Local path planning uses known local environment information for path planning. Local path planning has a high degree of flexibility and adjustment in

Authorized licensed use limited to: Frankfurt University of Applied Sciences. Downloaded on March 19,2026 at 17:13:34 UTC from IEEE Xplore. Restrictions apply.

face of dynamic and complex environment. However, because the prior information about the environment is not sufficient, the path has limitations and can only achieve local optimization. To this end, this paper proposed the improved A* algorithm for path planning and dynamic collision avoidance method.

II. Methodology

A. Improved A* algorithm

The priority function ***f(n)*** consists of the actual cost from node n to the starting point and the predicted cost from node n to the end point. The actual cost is generally not well adjusted, while the predicted cost is an estimate of the cost that can be adjusted. The predicted cost is a heuristic function that adds consideration of the end point to the path search and enables the algorithm to traverse towards the end point at a faster rate. If the estimated cost ***h(n)=0***, is not included, then the algorithm will be centered on the starting point and traverse in all directions until it reaches the end point. The Euclidean distance is set to different multiples and calculated, and the results shown in Table I are obtained.

TABLE I. Calculation Results For Different Multiples Of Euclidean Distance

h(n)	Table column subhead		
h(n)	***Iterate over the number of lattice points***	***Algorithmic efficiency***	***Path length***
0	2463	2.4%	65.21
Euclidean distance	586	10.0%	65.21
1.5 times Euclidean distance	95	66.3%	68.38
2 times Euclidean distance	70	87.1%	67.21
2.5 times Euclidean distance	63	93.7%	65.21

As the multiplier increases, the efficiency of the algorithm increases effectively and by a huge margin, but the path length is not necessarily the shortest and will increase slightly. This is because the increase in multiplier will make the search direction more inclined to the direction of the end point, but in this direction there will inevitably be obstacles. When searching for obstacles, the algorithm will only change the search direction, so the final path will appear to be a detour, the path length will increase. The choice of multiplier is related to the start point, the end point, and the obstacles in the whole map, by virtue of many calculations to take the optimal multiplier, it is found that taking 2.5 times Euclidean distance is the best.

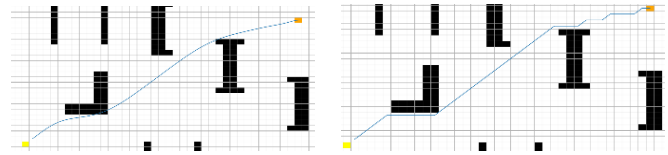
B. Method for smoothing paths

Given n+1 points $P_i(i = 0,1,\dots,n)$ in the plane, a Bessel curve of order n can be determined. The Bessel curve is determined by the following parametric equations:

$$B(t) = \sum_{i=0}^{n} P_i C_n^i t^i (1-t)^{n-i}, t \in [0,1]$$

Where t is the parameter, C_n^i is the number of combinations.

Under the optimization of the Bessel curve, the paths obtained are smoother, and a comparison of the paths before and after optimization is shown in Figure 1. The optimized curve may intersect with the obstacles, this is because the traditional A* algorithm will plan a path that sticks to the edge of the obstacles, after the Bessel curve optimization, the curve will become smoother but this makes the curve slightly shifted.



(a) Curve after optimization; (b) Curve before optimization

Figure 1: Schematic diagram of path comparison before and after Bessel curve optimization

According to the calculated smooth path, the robot has to advance along the path and complete the movement from the starting point to the end point [6]. Firstly, the path will be divided into several points, which will be used as sub-targets in the path tracking process of the robot, and then combined with the robot chassis control algorithm to control the robot to arrive at each sub-target in turn, so as to complete the whole path tracking process, as shown in Figure 2.

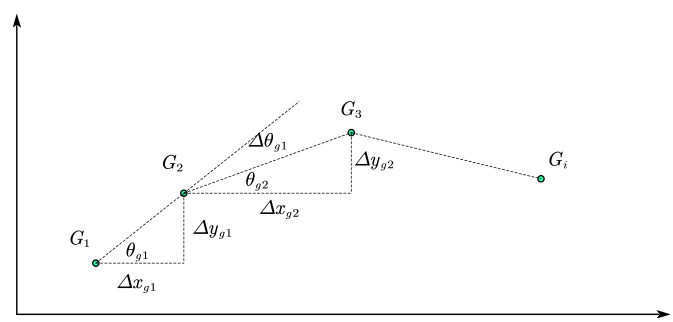


Figure 2: Schematic diagram of robot path tracking

Let the subgoals be $G_i(z_{yi}y_{yi})_z(i = 1,2,\dots,n)$ (in order from the start point to the end point), since the robot chassis control algorithm is concerned with relative coordinates and relative attitude, the relative coordinates and relative attitude between the subgoals are computed:

$$\begin{cases} \Delta x_{gi} = x_{g(i+1)} - x_{gi} \\ \Delta y_{gi} = y_{g(i+1)} - y_{gi} \\ \theta_{gi} = \arctan \dfrac{\Delta y_{gi}}{\Delta x_{gi}} \\ \Delta\theta_{gi} = \theta_{g(i+1)} - \theta_{gi} \end{cases}$$

For the convenience of subsequent obstacle avoidance, the robot chassis should be made to move forward as much as possible, the forward direction should be consistent with the path direction as much as possible. Therefore, the coordinates of the robot are set to be the same as the coordinates of the sub-target when it reaches the sub-target, and the direction of the robot is pointed to the next sub-target.

If Figure 2, G_1 is the starting point, and the position and orientation of the robot are set to be $(x_{g1}, y_{g1}, \theta_{rs})$, then the

Authorized licensed use limited to: Frankfurt University of Applied Sciences. Downloaded on March 19,2026 at 17:13:34 UTC from IEEE Xplore. Restrictions apply.

robot should be controlled to rotate $\theta_{g1} - \theta_{rs}$ so that its orientation is aligned with G_2, at this time, the coordinates G_2 under the robot coordinate system are changed to $\left(\sqrt{\Delta x_{g1}^2 + \Delta y_{g1}^2}, 0, \Delta\theta_{g1}\right)$, and the coordinates are brought into the motion control algorithm of the robot's chassis as a parameter that is to arrive G_2 at the sub-target, and the path tracking can be accomplished by repeating this step until the end point.

C. *Vision-Based Localization Algorithms*

The A* algorithm is able to plan a reasonable path given a start point, an end point, and a global map. The starting point is the current position of the robot, the end point is a human-specified location, and the global map is the realization of the given map. To perform path planning, the starting point, which is the current position of the robot, must first be determined.

The robot can recognize 47 types of visual labels, including numbers, letters and some special symbols. Each visual tag has a specific id number, and the robot returns the number information to the user during the recognition process. When the robot acquires the image received from the camera, it will do the following processing on the image and the principle of visual label recognition shown in Figure 3:

(1) RGB model is converted to HSV model and the red color is extracted

(2) Binarize the visual label image

(3) Extract the contour information of the binarized image

(4) Perform polygonal contour fitting to determine if it is a quadrilateral structure

(5) Obtain the R01 region

Figure 3: Principle of visual label recognition

For each tag, the robot saves the hash value of the recognized tag in its 4 rotational directions, so rotating the visual tag does not affect the recognition performance of the robot.

If no visual tags are recognized, the above steps do not return information about the visual tags and the robot informs the user that the number of visual tags in the field of view is 0. If visual tags are recognized, the robot informs the user how many visual tags are recognized and what the information about each of these visual tags is.

When a visual tag appears in the field of view and is recognized by the robot, the robot calculates and analyzes the information of the visual tag according to the recognition principle: the number of tags in the field of view, the id of the tag, the horizontal and vertical coordinates of the tag's center in the coordinate system of the field of view, and the width and height of the tag in the coordinate system of the field of view.

An algorithm is proposed to determine the position of the robot in the scene using the visual tag recognition function of the robot. As shown in Figure 4, a vision label with a width of 100 mm is attached to the wall directly in front of the robot.

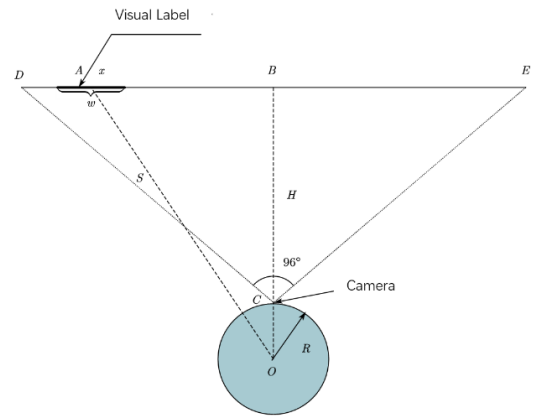


Figure 4: Schematic diagram of the robot in relation to the position of the vision tag (top view)

Assuming that the center of the vision tag is at a distance of S to the yaw axis of the robot, the camera is at a distance of R to the yaw axis of the robot, $R = 77.5mm$, the camera is at a distance of H from the wall, the center point of the vision tag under the pixel coordinate system is x at the coordinates, and the width is w. If the camera distortion is not considered, the size of the object under the pixel coordinate system and the actual size are similarly related, noting that the similarity ratio is k, and the size of the object under the pixel coordinate system is denoted by the corresponding lowercase letter.

Summarize the solution:

$$S = \sqrt{\left(\frac{100}{w}\frac{0.5}{\tan 48°} + R\right)^2 + \left(\frac{100}{w}(0.5 - x)\right)^2}$$

The attitude angle of the robot with respect to the visual label can also be derived:

$$\theta = \arctan\frac{k(0.5 - x)}{H + R} = \arctan\frac{k(0.5 - x)}{\frac{100}{w}\frac{0.5}{\tan 48°} + R}$$

One distance can be calculated from one vision tag, as well as the attitude of the robot. A plurality of distances can be calculated from a plurality of visual tags, and the plurality of distances combined can estimate the position and attitude of the robot.

Similarly, the side view can be analyzed as in Figure 5.

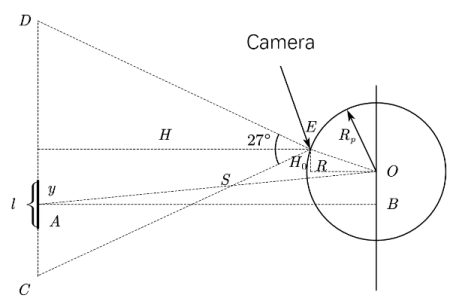


Figure 5: Schematic diagram of the robot in relation to the position of the vision tag (side view)

Authorized licensed use limited to: Frankfurt University of Applied Sciences. Downloaded on March 19,2026 at 17:13:34 UTC from IEEE Xplore. Restrictions apply.

Summarize the solution:

$$S = \sqrt{\left(\frac{100}{l}\frac{0.5}{\tan 27^\circ} + R\right)^2 + \left(\frac{100}{l}(y - 0.5) - H_0\right)^2}$$

Use of visual labels for localization. Certain known locations on the map are affixed with visual labels, and the robot will obtain a series of visual label information after recognizing the visual labels, and the current position of the robot can be calculated after processing the visual label information, the schematic diagram of robot localization as shown in Figure 6.

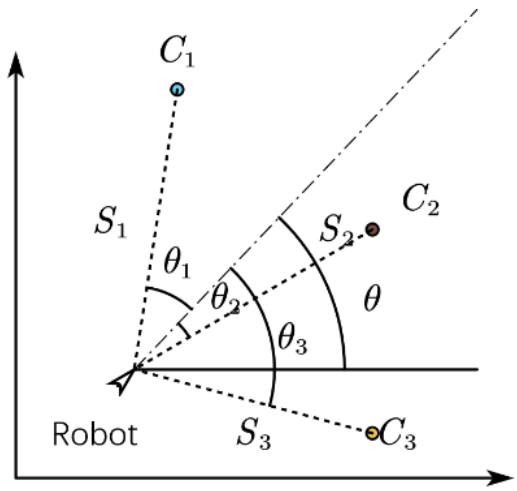


Figure 6: Schematic diagram of robot localization

First calculate the robot coordinates

$$(x - x_{ci})^2 + (y - y_{ci})^2 = S_i^2 (i = 1,2,\ldots,n)$$

Then calculate the robot attitude angle θ

$$\theta_i + \operatorname{atan}\frac{\hat{y} - y_{ci}}{\hat{x} - x_{ci}} = \theta$$

For each visual tag, one θ, which varies because of errors in the recognition calculations, can be computed, and all angles θ are averaged to obtain the final pose $\hat{\theta}$.

D. Dynamic Obstacle Avoidance Method Based on TOF Sensor

Due to the poor accuracy of the raster maps used in the A* algorithm, the existence of localization errors in the robot's odometers during operation, and visual errors caused by ambient lighting conditions, distance sensors are needed to achieve accurate dynamic obstacle avoidance.

In general, TOF distance sensors can acquire the distance between an obstacle and the sensor in a single direction. For a localized environment, it is necessary to know the distance between an obstacle and the robot in a range of directions [7]. For robot path planning, it only needs to detect the range around its forward direction. The TOF distance sensor of robot can be mounted on the gimbal, and as the gimbal rotates, the TOF sensor can rotate with it. By controlling the rotation of the gimbal's heading axis, sweeping through an angular range, and acquiring the TOF and gimbal heading axis angle information at the same time, the robot can acquire the sensor's information at a frequency of up to 50 Hz, and reasonably controlling the gimbal's rotation speed to acquire a sufficiently large number of sampling points, so that it can roughly detect the local environment within this range.

In general, directly sampled data is subject to error due to a number of factors, so the data is further processed to minimize the error.

An image is plotted in the polar coordinate system as in Figure 7, which shows the local environmental information detected by the robot.

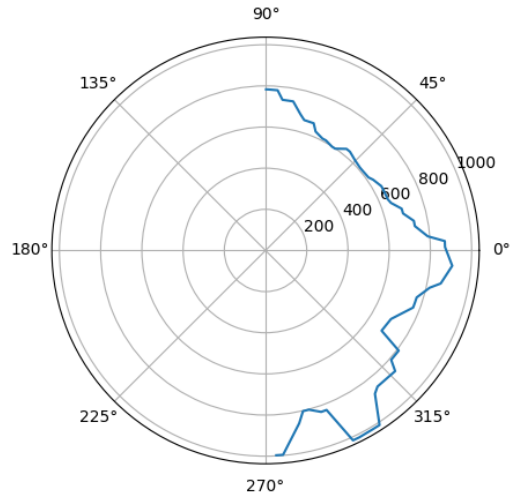


Figure 7: Schematic of the local environmental information detected by robot

To avoid the robot encountering obstacles during traveling, it needs to choose the traveling direction reasonably [8]. Through the detection of the surrounding environment, the obstacle information in the environment is obtained, according to which the traveling direction can be judged, as shown in Figure 8.

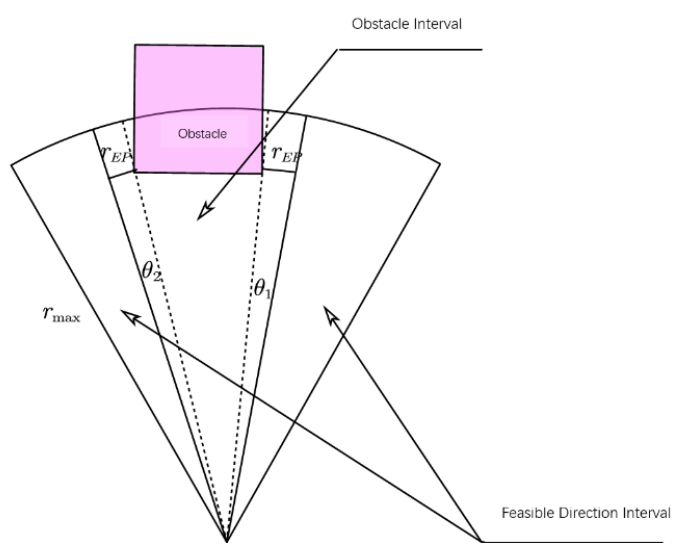


Figure 8: Schematic diagram of local environment information

The obstacle interval is $Q = Q_0 + Q_1 + Q_2.$, and the feasible direction interval is $P = P_0 - Q_1 - Q_2$.

Authorized licensed use limited to: Frankfurt University of Applied Sciences. Downloaded on March 19,2026 at 17:13:34 UTC from IEEE Xplore. Restrictions apply.

The direction of travel should be chosen in the feasible direction interval so as to ensure the requirement of safety without collision.

According to the improved A* algorithm, a continuous smooth path is obtained, and for the robot to move along the path, several points G_i on the path are taken as subgoals for the robot to move along. The robot can control the chassis to move to each subgoal with a certain position and attitude according to the chassis motion control algorithm. If the robot can follow the path smoothly, then it can accomplish the task of reaching the end point along the globally optimal path. If the robot can run smoothly and sequentially according to the given subgoals, then it can accomplish the task of reaching the end point along the globally optimal path [9,10]. If it encounters an obstacle on the way forward, the obstacle will be recognized by the TOF sensor on the robot, and at this time, the robot will adjust the direction of operation or adjust the subgoals. Specifically, it is categorized into the following scenarios, as shown in Figure 9.

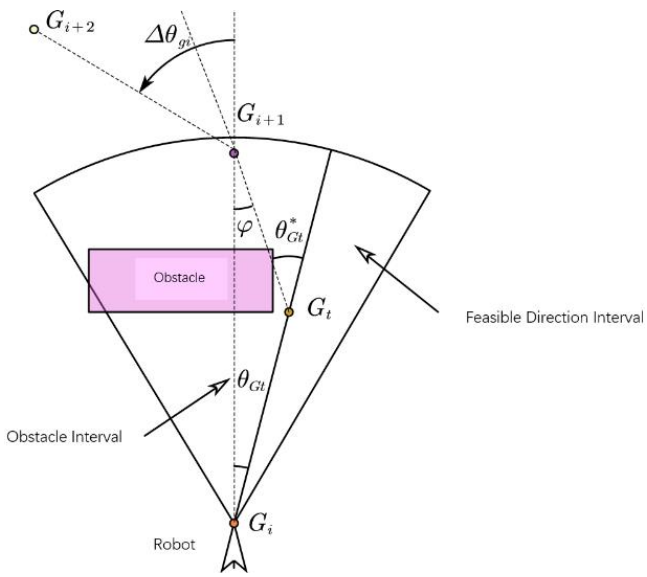


Figure 9: Schematic diagram of the obstacle between the robot and the sub-target

At this time, the robot must select the traveling direction from the feasible direction interval and take a point G_t in this direction as a temporary sub-target, and the robot reaches the temporary sub-target first and then reaches the sub-target G_{i+1}.

The control parameters of the subgoal are finally obtained. After arriving at the sub-target, if there is still an obstacle blocking, the above steps need to be repeated until there is no obstacle blocking. At this time to return to the sub-target G_{i+1}, a set of movement control parameters is also needed, and the necessary parameters $\left(G_t G_{i+1}, 0, \Delta\theta_{gi} - \varphi\right)$ have been obtained in the derivation.

The sub-target is inside the obstacle, as shown in Figure 10. Since the raster map used by the A* algorithm will have some errors, and obstacles that have not been considered before may also appear during the navigation process, there will be a situation where the sub-target is inside an obstacle.

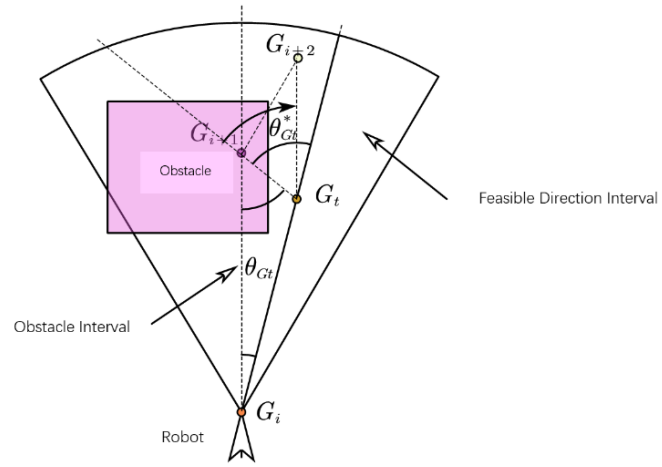


Figure 10: Schematic diagram of sub-target inside obstacle

In this case, the robot cannot reach the subgoal inside the obstacle no matter how it is adjusted, so it has to skip the subgoal and switch to the next subgoal.

Firstly, the robot will recognize that there is an obstacle in front of the subgoal that it cannot approach. Secondly, it will process according to the scenario of Case 1, and in the process, it will calculate θ^*_{Gt}, and if it is close to 90° then it can be determined that the sub-target is inside the obstacle, and then it will skip the sub-target and track the next sub-target.

The specific methods are:

(1) When the robot detects that the forward direction is blocked by an obstacle, calculate, to determine whether the sub-target is inside the obstacle or not.

(2) Calculate the distance from the subsequent subgoal node to the robot.

(3) If the distance is less than the set value, determine whether its travel direction is obscured by an obstacle and whether it is inside an obstacle.

(4) Take the sub-target node that satisfies the tracking requirements as the current sub-target node.

III. Results

A 38 x 30 lab site is built as shown in Figure 11. The site is labeled with visual labels 1, 5, and 4 posted in the figure where the numbers are labeled for the robot to compute its starting point, followed by a given point D as the end point of the robot's path planning. During the robot tracking process, 1 unknown obstacle is placed on its path to verify its dynamic obstacle avoidance function. The experimental results are presented as follows.

The robot is placed in region A in Figure 11, where visual labels 1, 4, and 5 can be observed, and its own position $(x, y, \theta) = (26.36, 2.52, 100.75)$ is calculated based on the visual labels, which basically conforms to the visually observed positional attitude. Subsequently, the robot calculates the path to the endpoint D according to the improved A* algorithm.

Authorized licensed use limited to: Frankfurt University of Applied Sciences. Downloaded on March 19,2026 at 17:13:34 UTC from IEEE Xplore. Restrictions apply.

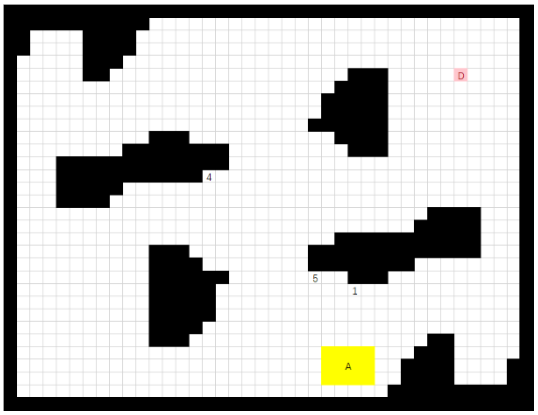
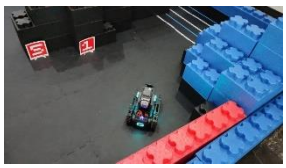
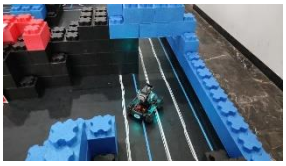
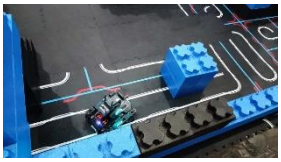


Figure 11: Schematic diagram of the experimental site

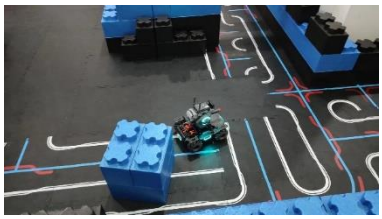
Then the robot moves along the subgoal points on the path and can actively avoid obstacles when it encounters them, thus avoiding collisions due to the fact that the path planned by the improved A* algorithm would overlap with the obstacles. After crossing the narrow path, a new obstacle appeared, and the robot recognized the new obstacle and bypassed it autonomously and reached the end point D successfully, as shown in Figure 12.

(a) Proceed from starting point (b) Avoid overlapping paths



(c) Encountering new obstacles (d) Avoiding obstacle



(e) Arrival at the end

Figure 12: EP robot path tracking video snapshot

IV. Conclusion and Discussion

This paper investigates the path planning, localization, and real-time obstacle avoidance problems of AGV robots. In order to meet the practical needs, a hybrid planning method is proposed, which first uses a vision-based localization algorithm for starting point localization, and then uses an improved A* algorithm for global path planning and real-time obstacle avoidance during path tracking. Specifically, the visual localization algorithm utilizes a geometric model that enables more accurate localization with the help of visual labels. The improved A* algorithm designs heuristic functions with weights. The real-time obstacle avoidance algorithm based on TOF sensors makes an obstacle avoidance action in response to local obstacle information. Experimental results show that this hybrid path planning algorithm effectively combines the A* algorithm and real-time obstacle avoidance and can meet the practical application requirements of AGV robots.

Future work will include autonomous map building of the environment with little or no reliance on visual labels and improve path tracking speed and efficiency of environment scanning.

References

[1] Y. Zhang, "Research on Robot Path Planning Based on Improved Genetic Algorithm," 2022 14th International Conference on Advanced Computational Intelligence (ICACI), Wuhan, China, pp. 273-27, 2022.

[2] Suresh, K., Venkatesan, R., & Venugopal, S. “Mobile robot path planning using multi-objective genetic algorithm in industrial automation. Soft Computing,” vol. 26, pp. 7387-7400, 2022.

[3] Y. Song and P. Ma, "Research on Mobile Robot Path Planning Based on Improved A-star Algorithm," 2021 International Conference on Electronic Information Engineering and Computer Science (EIECS), Changchun, China, pp. 683-687, 2021.

[4] F. Pan, “Research on Path Planning of AUV Based on Improved Genetic Algorithms,” International Conference on Computer, Control and Robotics, pp. 26-29, 2021.

[5] Y. Sun and H. Wang, A novel A* method fusing bio-inspired algorithm for mobile robot path planning. EAI Endorsed Trans. Scalable Inf. Syst., vol. 9, pp. e4, 2018.

[6] T. Sun, T. Wang and P. Sun, "Mobile Robot Dynamic Path Planning Based on Improved A* Algorithm," 2021 3rd International Conference on Robotics and Computer Vision (ICRCV), Beijing, China, pp. 24-29, 2021.

[7] Z. Liu, L. Gao, F. Liu, D. Liu and W. Han, "Fusion of weighted Voronoi diagram and A∗ algorithm for mobile robot path planning," 2022 2nd International Conference on Electrical Engineering and Mechatronics Technology (ICEEMT), Hangzhou, China, pp. 403-406, 2022.

[8] J. Ma, Y. Liu, S. Zang and L. Wang, "Robot Path Planning Based on Genetic Algorithm Fused with Continuous Bezier Optimization", Computational Intelligence and Neuroscience, vol. 2020, pp. 9813040, pp. 10, 2020.

[9] Wang, J. “Robot Path Planning Based on Artificial Intelligence Algorithm,” in 2021 International Conference on Big Data Analytics for Cyber-Physical System in Smart City. Lecture Notes on Data Engineering and Communications Technologies, vol 103, M. Atiquzzaman, N. Yen, Z. Xu, Eds. Springer, Singapore, 2022, pp. 927-933.

[10] Yang, H., & Teng, X. Mobile Robot Path Planning Based on Enhanced Dynamic Window Approach and Improved A∗ Algorithm. J. Robotics, vol. 2183229, pp. 1-9, 2022.

Authorized licensed use limited to: Frankfurt University of Applied Sciences. Downloaded on March 19,2026 at 17:13:34 UTC from IEEE Xplore. Restrictions apply.